

05 Agent 开卷速查

05 Agent 开卷速查

考场定位

本章按“AgentLab 涉及的 Agent 概念为主，补充 Agent 基本概念”准备。不要背工程命令，重点会解释：

- 为什么 Agent = LLM + Harness?
- 为什么要 loop / tools / memory / skill / trace / subagent?
- AgentLab 的设计分别对应哪些 Agent 概念？

考场小目录

题目	直接看	必写词
AgentLab 说明了什么	1. AgentLab 概念总表	LLM + Harness, JSON loop, tools, memory, skill, trace
Agent 是什么	2. Agent 基本定义	Agent = LLM + Harness
Agent loop	3. Agent Loop / ReAct	perceive/think/act/observe, reason + act
tool calling	4. Tools / Function Calling	intent, protocol, permission, observation
JSON 协议	5. Protocol / 结构化输出	tool_call, final, parse_error
memory	6. Memory	context compact, persistent memory, retrieval
skill	7. Skill / 经验库	摘要进 prompt, 正文按需加载

题目	直接看	必写词
subagent / multi-agent	<u>8. Subagent / Multi-Agent</u>	delegation, 分工, 并行, 校验
trace / evaluation	<u>9. Trace / 可观测性</u>	llm_response, tool_call, final
可靠性	<u>10. 可靠性问题</u>	hallucination, parse error, stale memory
简答模板	<u>11. 简答模板</u>	直接套句

来源优先级：ICS-AgentLab.pptx / AgentLab/README.md > 2-16-agent_*.pdf > review.pdf > ReAct / Toolformer / Voyager / Reflexion

1. AgentLab 概念总表

AgentLab 不是单纯项目说明，它把 Agent 的核心概念具体化：

AgentLab 设计	对应 Agent 概念	考场解释
普通 Chat Completion	LLM 只负责推理/生成	不依赖 SDK 自带 agent 能力
禁用 native tool calling	harness 自己控制工具协议	说明 Agent 不是 API 魔法，而是协议 + 调度
手写 JSON：tool_call / final	structured action protocol	LLM 输出行动意图，harness 解析执行
Agent.run_turn(...) 多轮循环	agent loop / ReAct	每轮 LLM -> action -> observation
read_file/write_file/edit_file/list_files/bash	tool use	让 Agent 能访问和改变外部环境
workspace 限制	permission / safety	工具执行必须有边界

AgentLab 设计	对应 Agent 概念	考场解释
load_skill	skill on-demand loading	流程说明不全塞 prompt，需要时再读
save_memory/read_memory	persistent memory / retrieval	跨 session 保存和读取稳定事实
context compact	short-term context management	当前对话太长时压缩，不等于长期记忆
ask_subagent	delegation / multi-agent	把边界清晰的小任务交给子 agent
trace	observability / evaluation	不只看最终答案，还看过程
token efficiency	context / prompt management	长任务中要控制 token 和工具次数

一句话总括：

AgentLab 考察的是 Agent 的 harness：用普通 LLM 加上手写协议、工具、memory、skill、subagent 和 trace，形成可控的任务执行循环。

2. Agent 基本定义

Agent = LLM + Harness

- LLM：理解任务、推理、规划、决定下一步。
- Harness：上下文、工具、权限、记忆、执行、日志、错误恢复。
- Goal：围绕目标持续推进，不是一次性聊天。
- Loop：感知 -> 推理 -> 行动 -> 观察 -> 再循环。

为什么不能只有 LLM：

LLM 只能产生文本/行动意图；真正完成任务还需要工具、状态、权限、执行和反馈闭环，这些都属于 harness。

五个展开点：

点	写什么
工具	读文件、shell、API、数据库
状态	历史消息、中间结果、失败重试
权限	workspace 限制、sandbox、越权控制
执行	把文本意图变成真实工具/API 调用
反馈	把结果回填上下文，进入下一轮推理

3. Agent Loop / ReAct

课件四步：

Perception -> Cognition -> Action -> Observation

AgentLab 里的实际形态：

LLM response -> parse JSON -> tool_call -> tool result -> append observation
-> next LLM response

步骤	通用概念	AgentLab 对应
Perception	读输入、规则、memory、环境状态	user input + system prompt + memory keys + tool results
Cognition	分析现状、定计划、决定下一步	LLM 生成 JSON
Action	调工具、写文件、跑命令、委托任务	tool_call / ask_subagent
Observation	收结果、错误、状态变化	tool result 写回 messages

ReAct：

Reasoning + Acting interleaving

- reasoning：跟踪目标、更新计划、处理异常。
- action：调用工具/访问环境。
- observation：把外部结果带回下一轮推理。

考场句：

Agent loop 让 LLM 不只是一次性回答，而是在 observation 反馈下不断调整行动；ReAct 强调 reasoning 和 acting 交错进行。

4. Tools / Function Calling

核心链路：

LLM 产出 action intent -> Harness 按协议解析 -> Tool 执行 -> Observation 回填

AgentLab 常见工具：

工具	概念作用
read_file	获取外部信息
write_file	改变环境 / 产出结果
edit_file	精确修改
list_files	探索环境
bash	执行命令
load_skill	按需加载流程
save_memory	保存长期事实
read_memory	检索长期事实
ask_subagent	委托子任务

为什么需要工具：

- LLM 内部知识不够、可能过时或不精确。
- 文件、命令、API 等外部状态必须实际访问。
- 工具结果作为 observation，能降低幻觉。
- Agent 要完成任务，往往需要改变环境，而不仅是说话。

工具使用四问：

问题	答法
何时调用	需要外部信息、精确结果、改变环境
调哪个	根据 tool description / schema
参数怎么传	按协议输出结构化参数
结果怎么用	作为 observation 更新答案或计划

5. Protocol / 结构化输出

AgentLab 禁用 SDK native tool calling，要求手写 JSON 协议。

典型格式：

```
{"type": "tool_call", "name": "read_file", "arguments": {"path": "a.txt"}}
{"type": "final", "content": "answer for the user"}
```

为什么需要 protocol：

- 让自然语言意图变成可执行动作。
- 让 harness 能解析工具名和参数。
- 限制 LLM 只能在允许格式内行动。
- 方便记录 trace 和做错误恢复。

常见错误：

错误	应对
输出 Markdown 代码块	强化“只输出 JSON”，解析时容错
JSON 不合法	记录 parse_error，让模型修复
工具名错	tool description / registry 保持一致
参数错	schema 清楚，错误反馈给模型
一直不行动	prompt 要求必要时调用工具

一句话：

Protocol 是 LLM 和 harness 之间的接口：LLM 输出结构化 action，harness 负责解析、执行和回填 observation。

6. Memory

AgentLab 最容易考的边界：

context compact != persistent memory

机制	负责什么	不该做什么
context compact	压缩当前 session 旧消息	不写入长期 memory
persistent memory	跨 session 的稳定事实/偏好	不保存临时噪声
retrieval	按 key/相关性取记忆	不要全部塞进 prompt

为什么 memory 要分层：

context window 容量有限；长期信息不能全部塞进 prompt，只能按需读取、必要时压缩、旧信息可替换或更新。

Memory 类型：

类型	例子
short-term memory	当前任务的计划、中间结果、tool observation
long-term memory	用户偏好、项目事实、跨 session 信息
episodic memory	做过哪些步骤、遇到什么失败
procedural memory	某类任务怎么做，接近 skill

AgentLab memory 要点：

- system prompt 只放 memory key 列表，不放全部 memory。
- 需要长期事实时调用 read_memory。
- 用户要求 remember/update/persist 时调用 save_memory。
- 更新后新事实优先。
- 没证据就说未知，不编造。

7. Skill / 经验库

Skill 是可复用流程，不是答案库。

AgentLab 机制：

设计	概念
skills/<name>/SKILL.md	skill 文档
front matter name/description	skill 摘要
system prompt 只放摘要	减少上下文
load_skill 读取全文	按需加载
skill 写 checklist / output requirements	稳定行动流程

Skill 和 memory 区别：

项	Memory	Skill
保存什么	事实、偏好、历史经验	可复用流程、checklist、操作规范
何时用	需要某个事实时	遇到某类任务时

项	Memory	Skill
作用	记住信息	指导怎么做

考场句：

Skill 是按需加载的任务流程。摘要放进 prompt，正文需要时再读，可以减少 token，并让 Agent 按 checklist 稳定执行。

8. Subagent / Multi-Agent

AgentLab 的 ask_subagent 对应 subagent / multi-agent delegation。

适用场景：

- 子任务边界清晰。
- 输入输出明确。
- 主上下文不想塞太多细节。
- 需要分工、并行、复核。

好处：

好处	写法
分工	不同 agent 负责不同子任务
并行	多个子任务同时推进
通信	agent 间交换中间结果
协调	主 agent 汇总和合并结果
互查	一个生成，一个检查
降低上下文复杂度	子 agent 只看局部信息

风险：

- token 成本上升。
- 角色不清会互相推诿。
- 子 agent 输出错误会污染主 agent。
- 需要明确停止条件和输出格式。

一句话：

Subagent 是把局部任务委托出去，主 agent 只接收整理后的结果；multi-agent 的关键是分工、通信、协调和校验。

9. Trace / 可观测性

AgentLab 强调 trace，因为评测不只看最终答案，也看过程。

至少记录：

- llm_response
- tool_call
- parse_error
- context_compacted
- memory_write
- memory_retrieve
- final

Trace 的作用：

作用	解释
可观测	知道 Agent 每一步做了什么
调试	定位 JSON、工具、memory、skill 问题
评测	证明真的调用工具，不是编造答案
效率分析	看 LLM 请求数、工具次数、token、压缩次数

一句话：

Trace 是 Agent 执行过程的证据链；复杂 Agent 不能只看 final answer，还要看中间行为是否正确。

10. 可靠性问题

常见问题和对应概念：

问题	表现	应对
hallucination	编造事实/工具结果	工具验证，引用 observation
parse error	输出不符合 JSON	固定协议，修复重试
over-planning	一直计划不行动	要求必要时调用工具
context overload	上下文太长，忽略重点	compact, retrieval
stale memory	旧记忆覆盖新事实	更新优先

问题	表现	应对
tool misuse	工具名/参数错	schema 清晰，错误反馈
infinite loop	重复失败	最大步数，reflection， 停止条件
token explosion	prompt/skill/tool result 太长	按需加载，压缩，限制输出

Reflection:

Reflection 是 Agent 根据失败轨迹或冲突 observation 自我检查、修正计划、更新经验，避免重复同一错误。

Token efficiency:

- prompt 简洁明确。
- skill 正文按需加载。
- tool result 不要过长。
- compact 保留目标、关键 ID、路径、证据、失败尝试、下一步。
- 不做无意义多轮解释。

11. 简答模板

11.1 AgentLab 说明了什么

AgentLab 说明 Agent 的核心是 LLM + Harness。LLM 负责推理和生成行动意图，harness 负责 JSON 协议、工具调度、权限控制、memory、skill、subagent、trace 和错误恢复。

11.2 Agent 是什么

Agent = LLM + Harness。LLM 负责理解、推理和决策，harness 负责上下文、工具、权限、记忆和执行反馈。它通过 perception、cognition、action、observation 的循环围绕目标持续完成任务。

11.3 为什么不能只有 LLM

因为 LLM 只能产生文本或行动意图，真正完成任务还需要工具执行、资源访问、权限隔离、状态维护、错误恢复和日志追踪，这些由 harness 负责。

11.4 tools / function calling

Tool calling 的关键是受控执行：LLM 产生 action intent, harness 按协议解析并检查权限，再执行工具并把结果回填为 observation。

11.5 为什么要手写 JSON 协议

JSON 协议把自然语言意图变成结构化 action，使 harness 能稳定解析工具名和参数，区分 tool_call 与 final，并在格式错误时记录 parse_error 进行恢复。

11.6 为什么 memory 要分层

因为上下文窗口有限，Agent 不能保留全部历史，只能把重要信息按需加载，对当前上下文压缩/替换，并把稳定事实持久化保存。context compact 不等于长期 memory。

11.7 skill 是什么

Skill 是可按需加载的流程说明或经验库。system prompt 只放名称和摘要，真正需要时加载正文，以减少 token 并让模型按 checklist 行动。

11.8 trace 有什么用

Trace 记录 llm_response、tool_call、parse_error、memory 操作和 final，用于调试、评测和证明 Agent 真的按步骤执行，而不是只编造最终答案。

11.9 subagent 有什么好处

Subagent 能把复杂任务拆成边界清晰的子任务，由不同角色并行推进并互相校验，从而降低主上下文复杂度，提高扩展性和鲁棒性。

12. 高频词

- Agent = LLM + Harness
- Perception -> Cognition -> Action -> Observation
- ReAct = reasoning + acting
- tool_call / final / parse_error
- protocol / permission / observation / trace

- context compact != persistent memory
- read_memory / save_memory / retrieval
- SkillLoader / load_skill / SKILL.md
- subagent / ask_subagent
- reflection / self-correction
- token efficiency / context compression

13. 考场最后 20 秒

1. 总纲: Agent = LLM + Harness
2. AgentLab: 手写 JSON loop + tools + skills + memory + trace
3. loop: Perception -> Cognition -> Action -> Observation
4. tools: intent -> protocol -> execute -> observation
5. protocol: tool_call / final / parse_error
6. memory: context compact != persistent memory
7. skill: 摘要进 prompt, 正文按需 load
8. subagent: 委托局部任务, 主 agent 汇总